

Projeto Loja Veloz - Relatório TécnicoLoja

Veloz - Plataforma de Pedidos em Microserviços

Docker • Kubernetes • CI/CD • Observabilidade

Aluno: Gustavo Alves

Disciplina: Cloud DevOps

Data: Maio/2026

Parte Teórica

Introdução

A Loja Veloz é uma empresa de varejo digital que precisava modernizar sua aplicação para suportar crescimento, reduzir riscos durante deploys e melhorar a observabilidade dos serviços.

Para resolver esse problema foi adotada uma arquitetura baseada em microserviços utilizando Docker, Kubernetes e práticas DevOps.

A aplicação foi dividida em três serviços principais:

- Serviço de Pedidos
- Serviço de Pagamento
- Serviço de Estoque

Essa separação permite que cada serviço seja desenvolvido, atualizado e escalado de forma independente.

Microserviços e DevOps

A arquitetura de microserviços consiste em dividir uma aplicação grande em pequenos serviços independentes.

As vantagens incluem:

- Maior escalabilidade
- Facilidade de manutenção
- Melhor organização do código
- Deploy independente dos serviços

O DevOps tem como objetivo integrar desenvolvimento e operações através da automação de processos, reduzindo erros e acelerando entregas.

Docker e Kubernetes

Docker é uma plataforma de containerização utilizada para empacotar aplicações juntamente com suas dependências.

No projeto foi utilizado Docker Compose para facilitar o desenvolvimento local.

Com apenas um comando todos os serviços podem ser iniciados:

```
docker compose up --build
```

Kubernetes é uma plataforma de orquestração de containers responsável por:

- Gerenciamento de Pods
- Escalabilidade
- Recuperação automática
- Balanceamento de carga
- Atualizações sem indisponibilidade

Enquanto Docker executa containers, Kubernetes administra esses containers em produção.

CI/CD

CI/CD significa Integração Contínua e Entrega Contínua.

Foi utilizada uma pipeline automatizada através do GitHub Actions.

A pipeline é responsável por:

- Build da aplicação

- Validação automática
- Testes
- Publicação de artefatos

Essa automação reduz erros manuais e melhora a qualidade das entregas.

Observabilidade

A observabilidade permite monitorar a aplicação e identificar problemas rapidamente.

Ela é composta por:

Métricas

Monitoramento de CPU, memória e utilização dos serviços.

Logs

Registro de eventos e erros gerados pelos microsserviços.

Traces

Rastreamento das requisições entre os serviços da aplicação.

Ferramentas sugeridas:

- Prometheus
- Grafana
- OpenTelemetry

Decisões Arquiteturais

As principais decisões adotadas foram:

- Utilização de microsserviços
- Docker Compose para desenvolvimento local
- Kubernetes para produção
- GitHub Actions para CI/CD
- Escalabilidade automática com HPA

- Estratégia de deploy Rolling Update

Essas decisões tornam a aplicação mais escalável, organizada e confiável.

Parte Prática

Ambiente Local com Docker Compose

Foi criado um ambiente multi-serviço utilizando Docker Compose.

Os serviços implementados foram:

- Pedidos
- Pagamento
- Estoque

Todos os serviços podem ser iniciados com:

```
docker compose up --build
```

Isso garante um ambiente reproduzível e padronizado para desenvolvimento.

Containerização

Cada microsserviço possui seu próprio Dockerfile.

As imagens são construídas de forma independente, permitindo:

- Facilidade de manutenção
- Padronização
- Portabilidade

Kubernetes

Foi realizada a implantação da aplicação em Kubernetes através de:

Deployments

Responsáveis pela criação e gerenciamento dos Pods.

Services

Responsáveis pela comunicação entre os microsserviços.

HPA (Horizontal Pod Autoscaler)

Configuração utilizada:

- Mínimo: 2 Pods
- Máximo: 5 Pods
- Escalabilidade baseada em CPU

Estratégia de Deploy

Foi utilizada a estratégia Rolling Update.

Benefícios:

- Atualização gradual
- Menor indisponibilidade
- Redução de riscos durante deploy

Pipeline CI/CD

Foi criada uma pipeline utilizando GitHub Actions.

Fluxo:

1. Push para o GitHub
2. Build automático
3. Validação do projeto
4. Publicação

Essa abordagem reduz erros e acelera entregas.

Observabilidade e Escalabilidade

A solução prevê:

Observabilidade

- Logs centralizados
- Métricas de desempenho
- Tracing distribuído

Escalabilidade

- HPA configurado
- Escalabilidade automática conforme carga

Conclusão

O projeto Loja Veloz demonstrou a utilização de práticas modernas de DevOps através da integração entre Docker, Kubernetes e CI/CD.

A solução proposta permite maior disponibilidade, facilidade de manutenção, escalabilidade automática e automação de deploys, atendendo aos requisitos de um ambiente cloud-native.

Links do Projeto

Repositório GitHub

<https://github.com/LuanHulne/API-VELOZ>

Vídeo Pitch

Referências

- Documentação Docker
- Documentação Kubernetes
- Documentação GitHub Actions
- Documentação Terraform
- Metodologia 12-Factor App
- Documentação OpenTelemetry
- Documentação Prometheus e Grafana